FilmDB 数据库性能实验

文件访问与关系数据库的性能及一致性比较

姓名： 学号：

01 主要结论

两库比较以端到端耗时、并发吞吐量与 P95 延迟为标准，条件为可全部放入内存且已预热的数据、 两库各限 8 GiB 并使用本实验设置的数据库参数（见第 05 节）。 100 倍串行操作中，PostgreSQL 的主键点查、全表子串搜索、分组聚合与批量姓名替换耗时更低，固定返回 54 行的人物连接则由 openGauss 略快； 10 倍规模、8 客户端并发下，PostgreSQL 的只读与混合负载吞吐量均较高，加入少量同步写以后两库的读延迟都上升。 据此，在本实验所测条件下优先选择 PostgreSQL。

文件与数据库的选择取决于访问方式与一致性要求。按编号读取单条静态记录时， 本文件程序在偏移索引已装载的短批次中平均耗时最低；若需要维护多表引用、 在操作失败后撤销已执行的修改，则应使用数据库，这两项能力本文件程序没有实现。

比较条件	主要结果
静态记录，按编号单条读取（索引已装载）	文件偏移索引 4.84 µs；PG 97.43 µs；OG 194.43 µs。
固定返回 54 行的关联查询	文件扫描 1814.34 ms；索引连接 PG 1.42 ms、OG 1.07 ms；三个规模的快慢方向一致，100 倍下 openGauss 更快。
批量字符串替换，同步完成更新	PG 1472.21 ms；文件 1827.72 ms；OG 3162.42 ms。
外键校验及事务失败后回滚	两库均拒绝非法引用，回滚后数据恢复原值。

本表的性能数字限定在 100 倍规模，PG 为 PostgreSQL 14.24、OG 为 openGauss 6.0.1。 点查为各轮平均单次耗时的三轮中位数，其范围见第 02 节；其余为三轮操作耗时的中位数，范围见第 03 节。 点查每轮 100 次、三轮使用同一随机序列，其中“文件偏移索引”一行的 100 次整批约 0.48 ms， 不含索引建立（0.946 s）与装载。其余操作不含数据装载与连接准备。 表中数字出自原实验；第 07 节的十万次点查与同库索引对照单独统计。

数据库如何处理一次失败的跨表操作

事务先把一部电影的片长从原值改为新值，再插入引用不存在电影的演职关系；两库均拒绝该插入， 执行回滚后先前成功的片长修改也被撤销。

检查步骤	PostgreSQL 与 openGauss 的实际结果
事务内读取片长	可见修改后的值，确认第一步已执行。
插入非法演职关系	外键错误 23503，拒绝不存在的电影编号。
回滚并重新读取	片长恢复，四张表的内容摘要均与操作前一致。

数据库统一执行引用约束与事务撤销；本文件程序用临时文件替换来同步写入，若要提供同样的跨表保证需要另行实现检查与恢复逻辑。该验证只覆盖这一失败场景，不等同于崩溃恢复，也不覆盖所有隔离级别。

02　访问路径对查询性能的影响

同一份 100 倍数据，按编号顺序扫描单条记录平均约 263 ms，装载偏移索引后每轮 100 次整批约 0.48 ms。差别来自访问路径而非数据本身：文件按编号读取时必须逐行扫描，索引把编号直接映射到文件偏移，查询时查表后 seek 并读取一条记录；数据库则用主键等值查找。关联查询同样比较了文件扫描与数据库索引访问， 结果反映这些具体实现的差异。

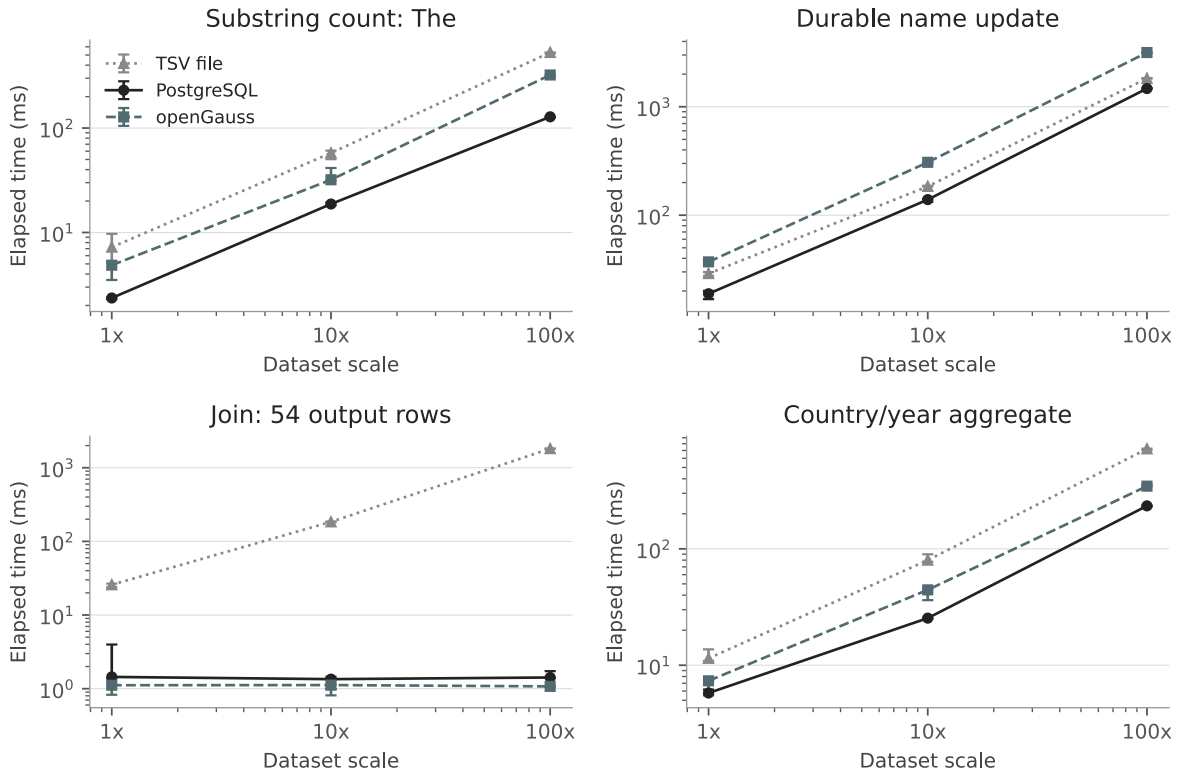


图 1　1 倍扩到 100 倍数据；子串计数与分组聚合需扫描全表，耗时随规模上升；人物连接中数据库索引连接的耗时基本平坦，文件端扫描则随规模显著增长。横纵轴均为对数刻度，误差线为三轮最小值至最大值。

人物连接中，文件程序依次扫描演职关系表与电影表；数据库先经人物索引定位演职记录，再用电影主键读取详情。 100 倍下文件端约 1814.34 ms，数据库索引连接约 1 ms。 三个规模都返回 54 行，且 1 倍与 10 倍下同样是 openGauss 略快于 PostgreSQL，方向与 100 倍一致； 因此这项测试考察的是固定结果量下的查询路径。

标题计数和分组聚合仍需扫描电影表。100 倍下 PG 聚合 234.02 ms， 约为文件程序的三分之一；这里同时存在数据格式和聚合实现的差异，不能把收益全部归因于数据库存储。

已装载索引下的单条记录读取

100×，按编号读取	平均单次 / µs	三轮范围 / µs
文件扫描	262765.74	[262640.25, 264006.09]
文件偏移索引	4.84	[4.67, 4.95]
PostgreSQL	97.43	[92.14, 98.34]
openGauss	194.43	[137.52, 195.71]

每轮 100 次，三轮使用同一随机序列；耗时列为每轮平均值的中位数，范围为各轮平均值的最小值与最大值。本表不含请求准备、连接及索引的建立或装载：文件索引整轮约 0.48 ms， 建立另需 0.946 s，因此“文件偏移索引”一行描述的是短批次、索引已装载的中位数； 五轮各十万次请求的重测见第 07 节。

标题搜索和人物连接没有使用文件偏移索引，需要各自的访问路径，不能套用本小节的数字。

03 扫描、更新与混合负载比较

在预热后的全表子串搜索、分组聚合及批量字符串替换中，100 倍规模下 PG 的端到端耗时均低于 OG。
下表保留文件实现作为参照；各项操作的规模与指标见表与表注。

100× 耗时 / ms	文件	PostgreSQL	openGauss
The 子串计数	529.23 [515.62, 529.85]	128.13 [126.06, 128.68]	322.35 [312.23, 324.21]
姓名同步更新	1827.72 [1819.72, 1836.73]	1472.21 [1471.99, 1481.63]	3162.42 [3132.07, 3169.66]
人物连接	1814.34 [1811.70, 1816.68]	1.42 [1.39, 1.73]	1.07 [1.06, 1.09]
国家 / 年份聚合	722.42 [703.17, 726.08]	234.02 [230.59, 234.67]	345.67 [345.26, 356.47]

三轮操作耗时的中位数，下行方括号内为［最小值，最大值］。 本轮属原 compact 批次（客户端绑定 CPU 16、17），与第 04 节的并发实验基线不同，两组数字不直接拼接比较。

标题搜索与结果集大小

100× 关键词	返回行数	文件 / ms	PG / ms	OG / ms
The	156,300	554.47	209.60	402.67
Love	8,800	528.57	125.68	313.70
Star	5,000	524.14	125.15	310.20

三组查询都扫描电影表，PG 均较快。同一后端内比较：PG 的 The 返回记录耗时 209.60 ms、仅计数 128.13 ms， 说明结果传输和处理也是端到端耗时的一部分；Love 和 Star 返回行数相差 3,800 行而耗时接近， 不同关键词还存在匹配计算差异，其耗时差不能全部归因于结果行数。

批量更新的端到端耗时

把姓名中的大写 T 替换成 Ttt，100 倍数据共修改 128,700 行。文件重写整张人物文件，同步后替换原文件； 数据库更新匹配记录并提交事务。PG 用时 1472.21 ms、OG 3162.42 ms， 后者也慢于文件端的 1827.72 ms。数据库无需重写整张人物表，但仍包含记录定位、修改与事务提交等开销；实验未分解两库的内部写入开销，因此不能仅凭结果确定差距来源。

读多写少负载的吞吐量与尾延迟

10×，8 客户端	PostgreSQL	openGauss
只读吞吐 / 万次·s^{-1}	9.41	7.35
混合吞吐 / 万次·s^{-1}	9.43	5.50
只读负载：读 P95 / ms	0.106	0.126
混合负载：读 P95 / ms	0.121	0.218
混合负载：写 P95 / ms	0.180	0.441

各项均取三轮中位数；P95 先在每轮内分别统计读、写请求，再取中位数，未合并三轮样本。 本轮属原 compact 批次。

混合负载约 90% 为主键读取、10% 为单行片长更新，每次更新单独同步提交。 从只读负载到混合负载，PG 的读 P95 由 0.106 升至 0.121 ms，写 P95 为 0.180 ms； OG 的读 P95 由 0.126 升至 0.218 ms，写 P95 为 0.441 ms。 加入少量同步写以后，两库的读延迟都上升；两库的差距来源未做进一步分解。

04 并发扩展与 CPU 瓶颈分析

在数据库绑定 8 核、客户端绑定 2 核的并发只读下，资源干预显示高并发时 PostgreSQL 的吞吐主要受客户端 CPU 限制， openGauss 主要受数据库 CPU 限制。 本节的 CPU 数由 `taskset` 指定绑定范围，不是独占预留， 服务器仍为共享使用，可能同时有其他任务；结论针对这一绑定配置，不是两库的处理上限。

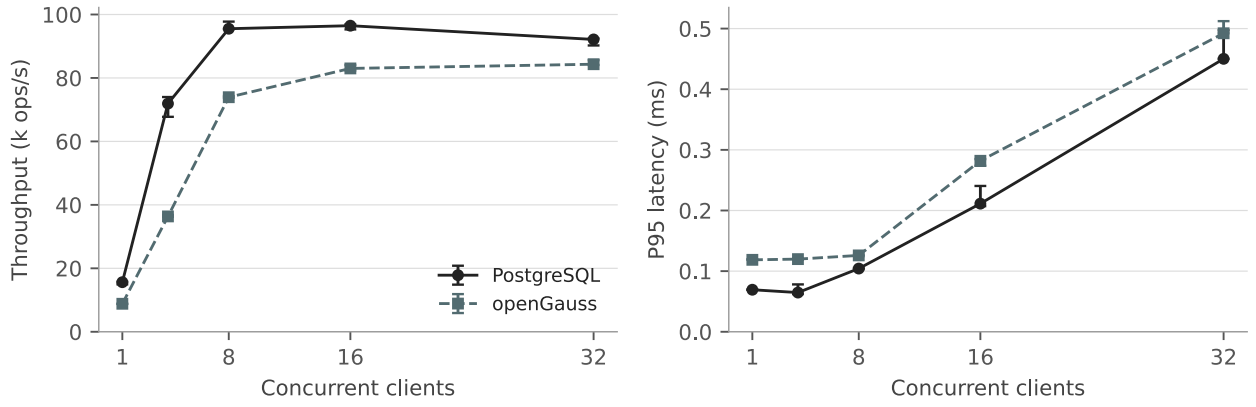


图 2 10 倍数据；数据库 8 核、客户端 2 核。每个客户端收到结果后才发送下一条请求。P95 为约 95% 请求能够完成的时间。

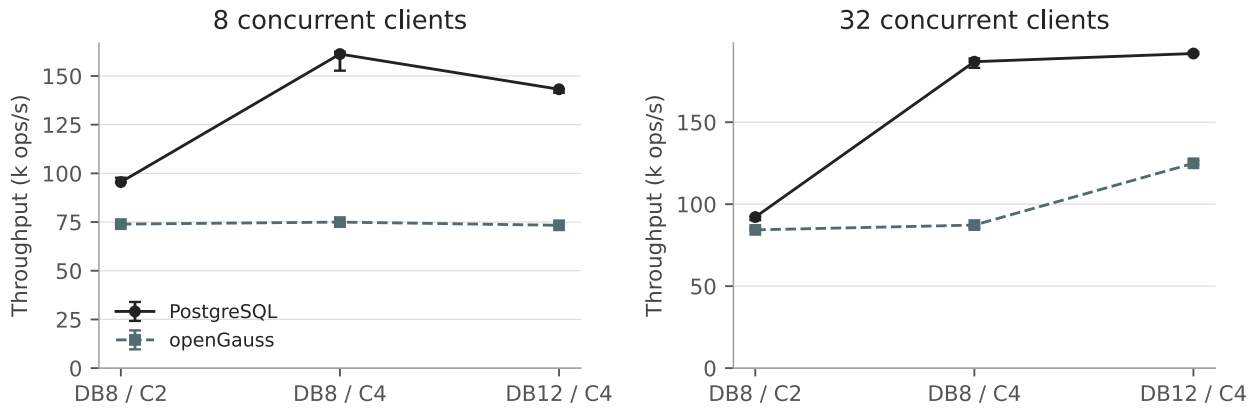


图 3 补实验分别改变客户端与数据库绑定的 CPU 数，查询、数据与数据库设置相同；DB 为数据库核数，C 为客户端核数。图 2 与图 3 取自同一批补实验、同一新基线。

并发从 1 增加到 16 时，PG 吞吐上升；到高并发后趋于平台，继续增加客户端未再提高吞吐，P95 延迟随之上升。 32 客户端纯读下，PG 客户端用了 2.00 核、数据库仅 4.06 核； OG 客户端 1.88 核、数据库 7.99 核，接近 8 核上限。 核数由每 0.5 秒采样的累计 CPU 时间增量算出，是剔除启动与预热后内部窗口内的近似平均忙核， 与吞吐量的测量窗口不完全重合，也不代表占用百分比。

32 客户端下，保持数据库 8 核、把客户端从 2 核增至 4 核，PG 吞吐由 9.21 升至 18.69 万次/秒 (逐轮配对比值中位数 2.02 倍)，OG 仅由 8.43 变为 8.72 万次/秒；保持客户端 4 核、把数据库增至 12 核，OG 升至 12.49 万次/秒（比值 1.45 倍)，PG 仅小幅升至 19.19 万次/秒。 把四个关键条件改为预热 10 秒、测量 120 秒、每条件单独一轮后， PG 的客户端扩展比为 2.03 倍、OG 的数据库扩展比为 1.42 倍，方向与原趋势一致； 该确认只有单轮，不提供波动范围。数据库与客户端各绑定 8 核的三轮附加组中，PG 达到 21.82 万次/秒、OG 8.51 万次/秒， 说明原配置下的吞吐平台不能直接视为数据库的处理上限。

05 实验设计与有效性范围

实验控制查询语义、输入数据和更新初始状态，以操作耗时、吞吐量及 P95 延迟评价性能， 以 CPU 核用量辅助分析资源瓶颈；结论限于可放入内存、已预热的数据。

数据与资源

课程 `filmdb.sql` 包含国家、电影、人物和演职关系四张表。扩容时复制后三表并平移编号，保持关联关系；国家表始终为 185 行。副本保留原字段分布，用 `replica` 区分重复标题和姓名。 100 倍数据的 TSV 合计约 161.1 MiB，可以全部放入内存。

规模	电影	人物	演职关系
1×	9,204	16,489	46,850
10×	92,040	164,890	468,500
100×	920,400	1,648,900	4,685,000

服务器为 Ubuntu 22.04、双路 AMD EPYC 7542、NVMe，同时运行其他任务。两库轮流运行，各限 8 GiB 内存， `shared_buffers` 为 512 MB、`work_mem` 为 4 MB，关闭单条查询内部的并行执行；文件程序、客户端与脚本共用 4 GiB。 基础配置把数据库与客户端分别绑定到 8 核与 2 核：原 compact 批次客户端为 CPU 16、17， 之后因该处邻核有其他任务，并发实验改用 CPU 24、25（或 24–27），仍与数据库同在 NUMA 节点 0 且不使用 SMT 兄弟。第 07 节沿用内存上限与数据库参数，但数据库改绑 CPU 9–16，测量期间逐秒记录 CPU、负载、内存与磁盘状态； 各批的客户端绑定与计时边界不同，因此各自只在本批内部比较，不把两批数字拼成一组对照。

测量与正确性

串行查询预热一次，并发先预热 5 秒、再测量 20 秒，关键 CPU 干预另用预热 10 秒、测量 120 秒的单轮确认。 通常每组重复三次并取中位数，误差线和范围为最小值至最大值；点查先在每轮内求平均，再取三轮中位数。 CPU 每 0.5 秒采样，P95 用桶宽约 1% 的直方图统计。查询耗时包括执行、结果接收和内容摘要计算； 标题搜索区分大小写，SQL 使用 `strpos(title,$1)>0`，文件使用 `string::find`。 姓名更新每轮前恢复原数据；两库开启 fsync、同步提交和 full_page_writes， 文件则先写临时文件、fsync，再替换原文件，使落盘条件接近。

原实验采用较早的源码：点查准备时间含扫描 TSV 收集候选编号，执行时间含随机选取编号及字符串转换。当前提交源码先生成完整请求序列，再开始查询计时，并拆分请求准备与后端准备（`timing_version=2`）；两版计时边界不同，本文保留原实验的实测值，不用当前源码的结果替换或反推。文件索引建立单独测量一次。

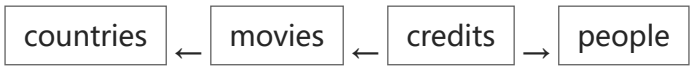
三个实现核对了查询内容、更新结果和表摘要。独立的 Python oracle 验证覆盖文本转义、NULL 与空串、 多角色演职、全 NULL 聚合、顺序/索引/SQL 点查一致性与陈旧索引拒绝；外键与回滚另由功能模式验证，见第 01 节。 此前功能验证的人物连接覆盖 0、1、22、54 行结果，均匀与热点两种点查各轮执行 10,000 次、重复 3 轮， 热点将 90% 请求集中在前 1% 编号；该轮服务器繁忙，记录标记为非性能测量，只作正确性证据。 上述通过不等同于验证了所有隔离级别或崩溃恢复。

实验局限

复制扩容没有引入新的数据分布，热缓存也没有展示大量磁盘读取的代价；短时测试和每轮恢复数据 未覆盖长期更新后的旧版本清理。此外，服务器资源非独占，并发采用闭环请求，三轮重复不足以确认微小差异。 原实验点查的窗口短且各轮使用同一序列，代表性有限，第 07 节以五轮各十万次请求、每轮独立序列重测点查， 并在同库内做人物索引对照；那批记录受共享服务器干扰，超出内存的数据与持续写入仍在本实验范围之外。

06 实现与复现

四张表通过外键连接；电影和人物通过演职关系形成多对多关系。同一人物可在一部电影中承担不同角色， 演职关系主键为 (movieid, peopleid, credited_as)。



箭头从外键所在表指向被引用的表：movies.country → countries.country_code；credits.movieid → movies.movieid；credits.peopleid → people.peopleid。

索引连接与分组聚合

人物索引以 peopleid 为首列，先定位演职记录，再通过电影主键读取详情；文件实现则扫描两张表。参数 $1 为人物编号。

```
CREATE INDEX credits_person_idx ON credits(peopleid, movieid);
SELECT m.movieid, m.title, c.credited_as
FROM movies m JOIN credits c ON m.movieid = c.movieid
WHERE c.peopleid = $1;
```

聚合按国家和年份分组，返回电影数、非空片长之和及其数量；文件端用 std::map 累计同样的结果，两库的诊断执行计划显示 HashAggregate。查询与建表语句内置于提交源码，数据导入另需 scripts/prepare.py 生成的四份 TSV。

姓名更新

筛选名字或姓氏包含大写 T 的人物，替换后提交。first_name 允许为空，筛选用 coalesce，更新保留其空值。 文件端使用同样的替换规则，重写人物文件。

```
UPDATE people
SET first_name = replace(first_name, 'T', 'Ttt'),
    surname = replace(surname, 'T', 'Ttt')
WHERE strpos(coalesce(first_name, ''), 'T') > 0
   OR strpos(surname, 'T') > 0;
```

代码与原始记录

主程序使用 C++17、libpq 和 pthread，内置建表 SQL。以下 Ubuntu 命令从固定仓库版本获取配套数据与脚本； 将本次提交的 filmbench.cpp 放入克隆目录后编译。 项目地址：github.com/luanmenglei/dbh_project1。

```
sudo apt-get install g++ libpq-dev python3 git
git clone https://github.com/luanmenglei/dbh_project1.git
cd dbh_project1
git checkout 668d1c787030c21b61bc062c55fca962ef4571d7
c++ -O3 -std=c++17 filmbench.cpp \
  -I"$(pg_config --includedir)" -L"$(pg_config --libdir)" \
  -Wl,-rpath,"$(pg_config --libdir)" -lpq -pthread -o filmbench
python3 scripts/prepare.py --scales 1
./filmbench --backend file --op search_rows \
  --data data/generated/s1 --keyword The
```

输出 rows=1563 表示匹配 1,563 部电影。数据库连接参数通过 FILMBENCH_DSN 环境变量传入； 以 --backend postgres --op load --data data/generated/s1 导入新建的默认实验 schema， 再将上例的后端改为 postgres 查询。openGauss 使用配套 libpq 编译，连接后选择 --backend opengauss。 详细部署与补测命令见 README。

性能数据固定在 experiments/baseline-20261012/ 与 results/ 的源码快照上；各次实际配置、逐轮原始输出、资源采样与源码哈希随结果目录保存，报告数字由构建脚本从这些记录审查导出： 若实验快照的哈希与记录不符、或提交源码与验证记录不符，构建会直接报错，不会静默替换。 当前提交源码另通过三种后端的正确性检查与功能验证，其计时边界与原实验源码不同（见第 05 节）。

07　长批次点查与人物索引对照

2026 年 10 月 15 日，用当前提交源码在同一台服务器上对 100 倍数据测量。数据库绑定 CPU 9–16、客户端绑定 CPU 24–25，同在 NUMA 节点 0。绑定范围只是 CPU 亲和性范围，不是独占配额： 测量与其他任务并行进行，整机 load 为 29.9–121.0， 仅按磁盘、内存与采样完整性排除轮次。

十万次请求下的点查

每轮生成一个新的请求序列，三种后端共用，均匀与热点分布各测五轮，每轮各十万次； 热点把 90% 请求集中在前 1% 编号。各后端先执行同一序列预热，再从请求序列就绪后开始计时， 请求文件的读取与序列生成不计入查询；五轮共三百万次正式查询，记录数与内容摘要逐轮一致；另有 60 次分层人物连接测量，覆盖不存在、低、中、高四档关联数量。

100×，后端	均匀 / µs	热点 / µs	后端准备 / s
文件偏移索引	3.29 [3.26, 3.47]	2.77 [2.74, 2.80]	0.380
PostgreSQL	63.38 [53.56, 88.19]	58.69 [49.37, 65.26]	0.012
openGauss	115.59 [112.79, 137.10]	115.25 [111.05, 158.92]	0.026

查询列为各轮平均单次耗时的五轮中位数，下行为最小值至最大值。准备列为均匀点查的后端初始化中位数： 文件端包括索引装载与校验，数据库端包括连接与语句准备；不含请求文件读取。文件索引另一次建立耗时为 1.026 s。 本批与第 02 节的短批次计时边界和 CPU 绑定都不同，两批不合并统计，也不据差值推算代码优化收益。

文件索引适合装载后反复访问静态记录：五轮各十万次查询中它仍是三种后端里最快的， 但装载索引本身约 0.380 s，只作少量查询时摊不掉这部分成本。

在同一数据库内切换人物索引

每库只导入一次数据，之后仅增删 `credits_person_idx`，五轮交替有 / 无索引状态，并交替数据库顺序。固定人物编号，每次返回 54 行；实验前后的四表内容摘要相同。每次先检查实际执行计划并预热，再测查询， 导入和索引切换均不计入查询时间。

100×，后端	有索引 / ms	无索引 / ms	逐轮加速比
PostgreSQL	1.43 [1.42, 1.60]	345.48 [344.34, 357.09]	243.3 [215.4, 243.9]
openGauss	1.24 [0.88, 1.30]	703.52 [626.05, 745.01]	566.9 [479.8, 845.8]

各列为五轮中位数与［最小值，最大值］。加速比先逐轮计算“无索引耗时 / 有索引耗时”，再取中位数， 不等同于表中两列中位数相除。两库五轮的实际计划均在有索引时使用人物索引，无索引时不使用该索引。 本对照不使用第 03 节的人物连接数字。

同一个固定 54 行的查询，同库加上人物索引把耗时从数百毫秒降到约 1 ms； 可见访问路径是这一查询效率的关键，结论限于该查询及其固定 54 行结果。

共享资源的限制

每个计时命令开始、结束及运行期间，每秒保存实验核和 SMT 兄弟核的 CPU 计数、负载与内存状态，覆盖准备与查询全过程。CPU 80 是数据库核 CPU 16 的兄弟核，两者共享物理核心， 该核在窗口内的平均忙碌比例为 46.0%，客户端核的兄弟核 CPU 88、 89 也有忙碌活动。忙碌比例不用于筛选样本，本批保留全部五轮及其范围； 核心共享与未隔离的缓存、内存带宽影响，使结果只适用于本批内部对照，不能推断独占环境下的性能。